REACT WEB

React 1 — Profile Card: полное руководство

Пошаговое учебное руководство
от условия до проверки и сдачи

Обязательная работа

Учебный комплект • 2026

> Статус: обязательная работа. Готовое решение находится в `projects/react-web/01-profile-card`.

Что требуется

Нужно создать компонент `ProfileCard`, который получает три props:

- `name: string` — имя;
- `age: number` — возраст;
- `showDetails: boolean` — флаг показа подробностей.

Имя отображается всегда. Возраст отображается только при строгом условии `showDetails === true`. В `App` должны находиться две карточки: одна с возрастом и одна без него.

В общей формулировке исходного задания упомянуты props и state, но конкретное проверяемое требование для этой работы — именно prop `showDetails`. Пользователь не изменяет этот флаг, поэтому искусственный `useState` здесь не нужен. Состояние без события, которое его меняет, не добавляет полезного поведения и только мешает объяснению решения.

Что получится

Итог — веб-приложение React + TypeScript + Vite. Это не React Native и не Expo.

Как отличить правильный проект

В корне правильного Vite-проекта находятся `package.json`, `index.html`, `vite.config.ts` и каталог `src`. Точка входа указана в `index.html` как `/src/main.tsx`, а компонент приложения находится в `src/App.tsx`.

Не создавайте `App.tsx` или `App.js` в корне репозитория. Не добавляйте `app.json`, Expo, `react-native`, `View`, `Text` или `StyleSheet`. Для этой работы используются DOM-элементы (`main`, `section`, `article`, `h1`, `h2`, `p`) и обычный CSS-файл.

Подготовка на macOS

Поддерживается Node.js `^20.19.4 || >=22.12.0`:

- Node 20 подходит начиная с 20.19.4 и до перехода на Node 21;
- Node 21 не подходит;
- Node 22 подходит начиная с 22.12.0;
- более новые версии подходят по диапазону проекта.

Проверьте окружение:

КОД · BASH

```
node --version
npm --version
git --version
```

Если Vite сообщает о неподдерживаемой версии Node.js, обновите Node.js. Удаление `package-lock.json` эту проблему не исправляет.

Создание с нуля

1. Откройте ссылку-приглашение GitHub Classroom и примите задание.
2. Дождитесь создания персонального репозитория и откройте его страницу.
3. Скопируйте HTTPS- или SSH-адрес репозитория и клонируйте его:

КОД · BASH

```
git clone <адрес-вашего-classroom-репозитория>
cd <имя-репозитория>
```

4. Убедитесь, что открыта правильная папка:

КОД · BASH

```
git status
ls
```

В выводе должны быть `package.json` и `src`. Если вместо этого вы видите Expo-файлы или корневой `App.tsx`, остановитесь: открыт не тот шаблон.

5. В клонированном Classroom-репозитории не заменяйте `package.json`, lock-файл, Vite- и TypeScript-конфигурацию готовыми файлами из другого проекта. Выполняйте задание только внутри `src`. Полные листинги ниже показывают точное состояние учебных файлов готового решения.
6. Восстановите зависимости:

КОД · BASH

```
npm ci
```

Чем отличаются npm install и npm ci

`npm install` читает диапазоны версий из `package.json`, подбирает разрешённые версии и при необходимости изменяет `package-lock.json`. Команда нужна при первом создании собственного проекта или при намеренном добавлении новой зависимости.

`npm ci` требует существующий `package-lock.json`, удаляет старый `node_modules` и устанавливает ровно зафиксированное дерево зависимостей. После клонирования Classroom-репозитория и перед сдачей используйте именно `npm ci`: преподаватель и студент получат одинаковое окружение.

Структура файлов

КОД · TEXT

```
01-profile-card/
├── index.html
├── package.json
├── package-lock.json
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.node.json
├── vite.config.ts
└── src/
    ├── App.css
    ├── App.test.tsx
    ├── App.tsx
    ├── main.tsx
    ├── ProfileCard.test.tsx
    ├── ProfileCard.tsx
    ├── vite-env.d.ts
    └── test/
        └── setup.ts
```

В Classroom меняются только файлы в `src`. `node_modules` и `dist` являются создаваемыми каталогами и не отправляются в Git.

Пошаговая реализация

Не соединяйте листинги в один файл. Заголовок каждого подраздела — точный путь относительно корня Vite-проекта.

src/ProfileCard.tsx

КОД · TSX

```tsx
export type ProfileCardProps = {
  name: string;
  age: number;
  showDetails: boolean;
};

export default function ProfileCard({
  name,
  age,
  showDetails,
}: ProfileCardProps) {
  return (
    <article aria-label={`Profile for ${name}`} className="profile-card">
      <h2>{name}</h2>
      {showDetails === true ? <p>Age: {age}</p> : null}
    </article>
  );
}
```

Что используется и зачем:

- `type ProfileCardProps` описывает внешний контракт компонента. TypeScript проверит тип каждого prop до запуска приложения.
- `name: string`, `age: number` и `showDetails: boolean` не позволяют случайно передать число вместо имени или строку `"false"` вместо boolean.

- `export` разрешает тестам и другим файлам импортировать тип; `export default` задаёт основной экспорт компонента.
- Параметр функции деструктурируется: `name`, `age` и `showDetails` сразу извлекаются из единого объекта props.
- Аннотация `: ProfileCardProps` связывает параметры с контрактом.
- JSX описывает DOM, который React должен создать.
- Семантический `article` представляет самостоятельную карточку. `className` — JSX-имя HTML-атрибута `class`.
- Шаблонная строка `` ` Profile for ${name} `` создаёт доступное имя карточки; `${name}` вставляет значение переменной.
- `<h2>{name}</h2>` всегда показывает имя. Фигурные скобки переключают JSX в JavaScript-выражение.
- Тернарный оператор проверяет `showDetails === true`. При строгом `true` создаётся абзац с возрастом, иначе React получает `null` и не выводит элемент.

src/App.tsx

КОД · TSX

```tsx
import './App.css';
import ProfileCard from './ProfileCard';

export default function App() {
  return (
    <main className="page">
      <h1>Profile cards</h1>
      <section aria-label="Student profiles" className="profile-grid">
        <ProfileCard name="Alice" age={20} showDetails={true} />
        <ProfileCard name="Ben" age={22} showDetails={false} />
      </section>
    </main>
  );
}
```

Что используется и зачем:

- `import './App.css'` подключает обычный CSS к графу модулей Vite.
- `import ProfileCard from './ProfileCard'` получает default export без указания расширения; Vite и TypeScript находят `.tsx`.
- `App` — функциональный React-компонент верхнего уровня.
- `main` задаёт основное содержимое страницы, `h1` — главный заголовок, `section` объединяет карточки.
- `aria-label="Student profiles"` даёт секции понятное доступное имя.
- Один и тот же компонент используется дважды с разными данными.
- Строка передаётся как `name="Alice"`, а числа и boolean — как JavaScript-выражения в фигурных скобках: `age={20}`, `showDetails={true}`.
- Здесь намеренно нет `useState`: оба флага заданы требованиями и не меняются во время работы приложения.

src/App.css

КОД · CSS

```
:root { font-family: Arial, sans-serif; color: #1f2937; background: #f8fafc; }
body { margin: 0; }
.page { max-width: 720px; margin: 0 auto; padding: 32px 16px; }
.profile-grid { display: grid; gap: 16px; grid-template-columns: repeat(auto-fit, minmax(220px,
1fr)); }
.profile-card { border: 1px solid #cbd5e1; border-radius: 8px; background: #ffffff; padding: 16px;
}
.profile-card h2 { margin-top: 0; }
```

Что используется и зачем:

- `:root` задаёт общий шрифт и цвета документа; второй шрифт `sans-serif` служит запасным.
- `body { margin: 0; }` убирает стандартный внешний отступ браузера.
- `.page` ограничивает ширину, центрирует содержимое через `margin: 0 auto` и добавляет безопасные внутренние отступы.
- `.profile-grid` включает CSS Grid. `gap` задаёт расстояние между карточками.
- `repeat(auto-fit, minmax(220px, 1fr))` автоматически выбирает число колонок: каждая не уже 220 px и может занимать свободную долю строки.
- `.profile-card` задаёт рамку, скругление, белый фон и внутренний отступ.
- Селектор `.profile-card h2` применяется только к заголовкам внутри карточки и убирает лишний верхний отступ.
- Это веб-CSS. Объект `StyleSheet.create` здесь не нужен.

src/main.tsx

КОД · TSX

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';

const rootElement = document.getElementById('root');

if (rootElement === null) {
  throw new Error('Root element #root was not found.');
}

createRoot(rootElement).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

Что используется и зачем:

- `StrictMode` включает дополнительные проверки React в режиме разработки.
- `createRoot` — браузерная точка входа React 18/19 из `react-dom/client`.
- `document.getElementById('root')` ищет контейнер, который уже объявлен в корневом `index.html`.
- DOM-поиск может вернуть `null`; явная проверка с `throw new Error` одновременно даёт понятную ошибку и сужает тип для TypeScript.

- `createRoot(rootElement).render(...)` связывает React с найденным DOM-элементом и рендерит `App`.
- Завершающая запятая допустима в многострочном вызове функции и упрощает автоматическое форматирование.

src/test/setup.ts

КОД · TS

```ts
import '@testing-library/jest-dom/vitest';
```

Что используется и зачем:

- Это импорт ради побочного эффекта: локальное имя не создаётся.
- Модуль добавляет к `expect` DOM-матчеры, в том числе `toBeInTheDocument` и `toHaveTextContent`.
- `vite.config.ts` готового репозитория подключает этот файл перед каждым тестовым файлом.

src/ProfileCard.test.tsx

КОД · TSX

```tsx
import { render, screen } from '@testing-library/react';
import ProfileCard from './ProfileCard';

test('always shows the name and shows age when showDetails is true', () => {
  render(<ProfileCard name="Mia" age={21} showDetails={true} />);

  expect(screen.getByRole('heading', { name: 'Mia' })).toBeInTheDocument();
  expect(screen.getByText('Age: 21')).toBeInTheDocument();
  expect(screen.getByRole('article', { name: 'Profile for Mia' })).toBeInTheDocument();
});

test('does not display an age when showDetails is false', () => {
  render(<ProfileCard name="Noah" age={34} showDetails={false} />);

  expect(screen.getByRole('heading', { name: 'Noah' })).toBeInTheDocument();
  expect(screen.getByRole('article', { name: 'Profile for Noah' })).not.toHaveTextContent(/Age:/);
});

test('does not display an age for a truthy non-boolean showDetails prop', () => {
  // @ts-expect-error Проверяем защиту от некорректного JavaScript-вызова во время выполнения.
  render(<ProfileCard name="Iris" age={29} showDetails="true" />);

  expect(screen.getByRole('article', { name: 'Profile for Iris' })).not.toHaveTextContent(/Age:/);
});
```

Что используется и зачем:

- `test(description, callback)` объявляет независимый сценарий Vitest.
- `render` создаёт компонент в виртуальном браузерном DOM, а `screen` ищет элементы так, как их воспринимает пользователь.
- `getByRole('heading', { name: 'Mia' })` ищет заголовок по семантической роли и доступному имени.
- `getByText` проверяет точный видимый текст возраста.
- `expect(...).toBeInTheDocument()` утверждает, что элемент присутствует.
- `.not.toHaveTextContent(/Age:/)` использует отрицание `not` и регулярное выражение, чтобы возраст не появился даже как другая цифра.

- `@ts-expect-error` фиксирует намеренную ошибку типов в одном тесте: TypeScript обязан отклонить строку, а runtime-проверка подтверждает, что строгий `=== true` не принимает truthy-строку.

src/App.test.tsx

КОД · TSX

```tsx
import { render, screen } from '@testing-library/react';
import App from './App';

test('renders exactly one detailed card and one name-only card', () => {
  render(<App />);

  expect(screen.getByRole('heading', { name: 'Profile cards' })).toBeInTheDocument();
  expect(screen.getByRole('article', { name: 'Profile for Alice' })).toHaveTextContent('Age: 20');
  expect(screen.getByRole('article', { name: 'Profile for Ben' })).toHaveTextContent('Ben');
  expect(screen.getByRole('article', { name: 'Profile for Ben' })).not.toHaveTextContent(/Age:/);
  expect(screen.getAllByRole('article')).toHaveLength(2);
});
```

Что используется и зачем:

- Здесь рендерится весь `App`, поэтому тест проверяет не только отдельную карточку, но и правильные props обоих экземпляров.
- `toHaveTextContent('Age: 20')` подтверждает подробную карточку Alice.
- Отрицательная проверка Ben подтверждает ложную ветвь условия.
- `getAllByRole('article')` возвращает все карточки, а `toHaveLength(2)` закрепляет требуемое количество.

src/vite-env.d.ts

КОД · TS

```ts
/// <reference types="vite/client" />
```

Что используется и зачем:

- Тройная директива TypeScript подключает типы клиентского окружения Vite.
- Благодаря ей TypeScript знает специальные возможности Vite, например типы `import.meta.env`.
- Файл не содержит runtime-кода и обычно уже создан шаблоном `react-ts`; не удаляйте его.

Порядок выполнения от пустого решения до готового

1. После `npm ci` запустите тесты один раз и зафиксируйте исходное состояние:

КОД · BASH

```bash
npm run test:run
```

2. Создайте `src/ProfileCard.test.tsx` и `src/App.test.tsx` по полным листингам. До реализации новые проверки могут завершаться ошибками — это ожидаемый этап.
3. Создайте `src/ProfileCard.tsx`.
4. Полностью замените содержимое `src/App.tsx` и `src/App.css`.
5. Сверьте `src/main.tsx`, `src/test/setup.ts` и `src/vite-env.d.ts` с листингами, не перенося их в корень.

6. Исправляйте первую ошибку, которую сообщает TypeScript или тест, и повторяйте проверку.

Запуск и тесты

Из корня проекта последовательно выполните:

КОД · BASH

```
npm run typecheck
npm run test:run
npm run build
```

- `typecheck` запускает `tsc -b` и проверяет контракты без генерации JavaScript-файлов рядом с исходниками.
- `test:run` запускает Vitest один раз; в готовом решении проверяются четыре сценария в двух файлах.
- `build` повторяет TypeScript-проверку и создаёт production-каталог `dist` средствами Vite.

Для повторного запуска тестов при каждом сохранении используется watch-режим:

КОД · BASH

```
npm test
```

Dev server и Preview

Запустите режим разработки:

КОД · BASH

```
npm run dev
```

Vite обычно печатает адрес `http://localhost:5173`. Изменения в `src` появляются почти сразу благодаря Hot Module Replacement. Остановите сервер сочетанием Control+C.

После успешной сборки отдельно проверьте файлы из `dist`:

КОД · BASH

```
npm exec -- vite preview
```

Preview обычно использует `http://localhost:4173`. Dev server на 5173 читает исходники и помогает разрабатывать; Preview на 4173 раздаёт уже собранный результат и не заменяет `npm run build`.

В готовом проекте на macOS/Linux также доступен автономный сценарий:

КОД · BASH

```
bash build-and-run.sh --build-only
bash build-and-run.sh --no-open
```

Он проверяет допустимую версию Node.js, устанавливает зависимости при необходимости, выполняет typecheck, тесты и сборку, а во втором режиме запускает Preview.

Ручная проверка

1. Откройте адрес dev server.
2. Найдите заголовок `Profile cards`.
3. Убедитесь, что на странице ровно две карточки.
4. В карточке Alice должны быть `Alice` и `Age: 20`.
5. В карточке Ben должно быть имя `Ben`, но не должно быть текста `Age:`.
6. Сузьте окно: сетка должна перестроиться без горизонтальной прокрутки.
7. В инструментах разработчика убедитесь, что нет runtime-ошибок.

Отправка в GitHub Classroom

Сначала проверьте список изменений:

КОД · BASH

```
git status
git diff -- src
```

В Classroom добавляйте только выполненные файлы из `src`:

КОД · BASH

```
git add src
git commit -m "Выполнить задание Profile Card"
git push
```

Не добавляйте `node_modules`, `dist`, `.env` и файлы Expo. После push откройте репозиторий на GitHub и убедитесь, что последний коммит появился в вашей ветке.

Типичные ошибки

- `showDetails="false"` передаёт строку. Правильно: `showDetails={false}`.
- `{showDetails && ...}` принимает любое truthy-значение; текущее решение намеренно использует строгое `showDetails === true`.
- `Age: age` выводит слово `age`; значение вставляется как `{age}`.
- Компонент с маленькой буквы React воспринимает как HTML-тег; имя должно начинаться с `ProfileCard`.
- `App.tsx` в корне, `StyleSheet.create`, `View` и `Text` означают, что смешаны Vite и Expo.
- Искусственный `useState(true)` без setter-события не выполняет дополнительного требования и усложняет код.
- Пустая страница часто означает неверный импорт, отсутствие default export или ошибку в `main.tsx`; читайте первую ошибку в терминале и консоли браузера.

Вопросы и ответы

Почему не используется `useState`? Конкретный флаг приходит через prop и не изменяется пользователем. Искусственное состояние без события не добавляет поведения.

Почему нужен строгий `showDetails === true`? Так правило принимает только boolean `true`, но не случайные truthy-значения вроде строки `"true"`.

Почему файлы лежат в `src`? Vite строит граф приложения от `/src/main.tsx`; корневой `App.tsx` относится к другому шаблону и не будет частью этого решения.

Чем Preview отличается от dev server? Dev server на 5173 работает с исходниками и HMR, а Preview на 4173 раздаёт результат уже выполненной production-сборки.

Защита

1. Почему props объединены в `ProfileCardProps` и какие типы там используются.
2. Почему имя видно всегда, а возраст создаётся только в одной ветви тернарного оператора.
3. Чем boolean `{false}` отличается от строки `"false"`.
4. Почему для этого конкретного задания не нужен state.
5. Как `App` повторно использует один компонент с разными props.
6. Чем отличаются typecheck, тест, build, dev server и Preview.

Чек-лист сдачи

- ☐ Открыт Vite-проект, а не Expo-проект.
- ☐ Все изменённые исходники находятся внутри `src`.
- ☐ `ProfileCard` принимает `name`, `age`, `showDetails` правильных типов.
- ☐ Имя отображается всегда.
- ☐ Возраст отображается только при `showDetails === true`.
- ☐ `App` создаёт две карточки с флагами `true` и `false`.
- ☐ Искусственное состояние не добавлено.
- ☐ `npm run typecheck` завершён без ошибок.
- ☐ `npm run test:run` завершён без ошибок.
- ☐ `npm run build` создал `dist`.
- ☐ Dev server проверен на порту 5173, Preview — на порту 4173.
- ☐ В commit не попали `node_modules`, `dist` и Expo-файлы.